DAPR AGENTS

AI agents that survive production

What it is, what it’s for, and a 2-minute demo with code.

[Your name] · [Event / team] · October 2026

WHAT IT IS

A Python framework for AI agents built on top of Dapr

1

Agent = code

LLM + instructions + tools + memory, in one class. You write Python functions; the model decides when to call them.

2

Infra = Dapr

State, workflow, pub/sub, secrets and the LLM itself are sidecar building blocks. Swapping a provider is YAML, not code.

3

CNCF project

Open source, maintained by the Dapr project. Same foundation that has run microservices in production for years.

WHAT IT'S FOR

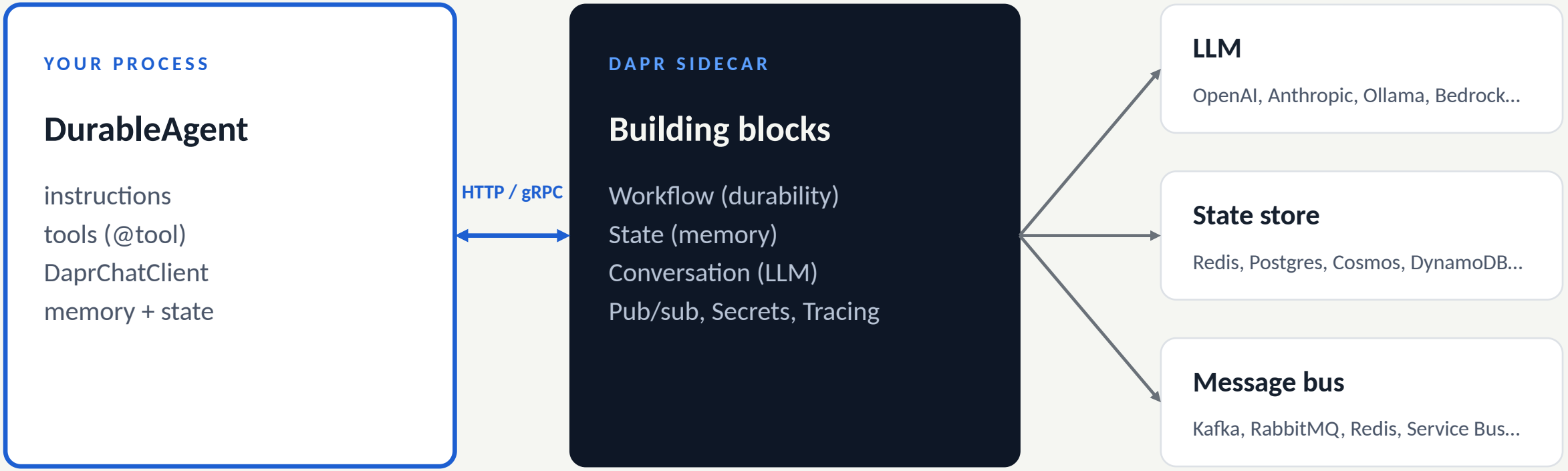
An agent in a notebook works. An agent in production breaks.

Without Dapr Agents	With Dapr Agents
Process dies mid-way through a 30s tool call: everything is lost, start over	**Durable execution**: every step is a workflow checkpoint; resumes where it stopped
Conversation history lives in RAM	**Memory in a state store**: survives restarts and scales horizontally
Each LLM vendor SDK hardcoded in the app	**Conversation API**: OpenAI, Anthropic, Ollama, Bedrock via YAML
Agents call each other over raw HTTP, no retries	**Pub/sub + resiliency**: queues, retries and timeouts from Dapr
Zero visibility into what the agent did	**Tracing and metrics** per step, for free, via OpenTelemetry

The pain is not the LLM. It's everything around it.

HOW IT WORKS

Your app talks to the sidecar. The sidecar talks to the world.



Every box on the right is a YAML component. Swapping one = editing the YAML.

USE CASES

Where a durable agent pays for itself

1

Customer support with real actions

Looks up the order, opens a ticket, issues a refund. Each action hits a slow external system; none may run twice. (Today's demo.)

2

Long-running back-office workflows

Claims, onboarding, invoice approval: minutes to days, with a human in the loop. Workflow waits for the event without burning a pod.

3

Multi-agent pipelines

Triage agent routes to expert agents over pub/sub. Each one scales, retries and is traced on its own.

4

Event-driven automation

An agent subscribed to a Kafka topic: new alert, new PR, new lead. Reacts, calls tools, publishes the result.

Common thread: slow steps, side effects, and a process that can die in between.

DEMO · THE CODE

One tool, one agent, one runner

```python
@tool
async def get_order(order_id: str) -> str:
    """Look up status and ETA of an order."""
    return ORDERS.get(order_id, "not found")

agent = DurableAgent(
    name="SupportAgent",
    role="Order support assistant",
    instructions=["Use tools, never invent data."],
    tools=[get_order, open_ticket],
    llm=DaprChatClient(component_name="llm-provider"),
    memory=AgentMemoryConfig(store=ConversationDaprStateMemory(
        store_name="agent-memory")),
    state=AgentStateConfig(store=StateStoreService(
        store_name="agent-workflow")),
)
AgentRunner().serve(agent, port=8001)
```

`@tool`
Docstring + type hints become the schema the LLM sees.

`llm-provider`
A Dapr component name. Which model? That's in the YAML.

`memory / state`
Two state stores: chat history, and workflow checkpoints.

`serve`
POST /agent/run returns an instance id; GET /agent/instances/{id} returns the result.

Full code: demo/agent.py and demo/tools.py (about 60 lines total)

DEMO · LIVE

Run it, ask it, kill it, watch it finish anyway

1 · RUN

```
dapr run --app-id support-agent
  --resources-path resources
  -- python agent.py
```

Sidecar starts, loads the YAML components, app listens on 8001.

2 · ASK

```
POST /agent/run
{"task": "Where is my
  order 1042?"}
```

Returns an instance id. LLM picks get_order, calls it, answers. GET /agent/instances/{id} shows every step.

3 · KILL IT

```
"Open a ticket for 1042"
^C   # during the 8s tool
dapr run ...   # restart
```

Same instance id: resumes at the tool step. The LLM call that picked the tool is not repeated.

Pre-recorded: demo/demo.mp4 (65s) · live: ./demo.sh run · ask · ticket · wait

TAKEAWAY

Write the agent. Let Dapr keep it alive.

Durable

Every LLM and tool step is a checkpoint.

Portable

LLM, state, queue: swap the YAML, keep the code.

Observable

Tracing per step, out of the box.

github.com/dapr/dapr-agents · pip install dapr-agents · [your contact]